ReMix: Reparameterization for Mixtures

A Blog Post on Gradient Estimation for Mixture Distributions

Alexander Udeneev, Alexey Ponomarev, Lina Gorbunova, Maxim Ivanov
IAS, MIPT

October 2026

Abstract

Finite mixtures are essential probabilistic tools capable of representing complex multimodality and heterogeneous uncertainties. However, variational inference with mixtures requires differentiating expectations with respect to both component parameters and mixture weights, where standard score-function estimators suffer from high variance and naive pathwise differentiation fails. In this blog post, we introduce **ReMix**, a modern Python framework built on PyTorch that unifies unbiased gradient estimators for mixture distributions, starting from our core prototype implementing the Score-Function with Sticking the Landing (STL) modification.

1 Introduction and Motivation

Mixture distributions take the form:

$$q_\theta(z) = \sum_{k=1}^{K} \pi_k(\theta) q_{k,\theta}(z), \quad \pi_k > 0, \quad \sum_{k=1}^{K} \pi_k = 1, \tag{1}$$

where $\theta \in \mathbb{R}^m$ parameterizes the density on $\mathbb{R}^d$. While sampling from a mixture is straightforward-first drawing a categorical component index and then sampling from that specific component-differentiating through this sampling process is notoriously tricky. Differentiating only the selected component ignores the vital contribution from changes in the selection probabilities (π_k).

In our project **ReMix**, we address this gap by designing a clean, extensible library architecture for mixture gradient estimation. Our goal is to support multiple unbiased estimators (score-function, stratified, implicit reparameterization, and post-stratified) under a single consistent PyTorch interface.

2 Planned Structure of the Post and Library

To provide a comprehensive overview of our work, our documentation and blog series follow this planned structure:

- **Section 1: Abstract & Motivation** - Introduction to mixture distributions and the gradient estimation problem in variational inference.
- **Section 2: The ReMix Interface Architecture** - How users define mixtures, objectives, and estimators in PyTorch.
- **Section 3: Score-Function Baseline and STL** - Mathematical foundations and autograd construction of the Score-Function with Sticking the Landing.
- **Section 4: Advanced Estimators (Stratified, IR, and Post-Stratified)** - Overview of low-variance pathwise alternatives.
- **Section 5: Reproducible Benchmark & Results** - Experimental evaluation on Gaussian and Laplace mixture fitting.

3 Planned Visualizations and Figures

Figure 1 (Main Visual / Architecture Schema):
A schematic flow diagram illustrating the software architecture: `Mixture (sample, log_prob)` → `ScoreFunction.estimate(q, objective)` → `EstimateResult (value, surrogate_loss)` → `backward()` via `torch.optim.Adam`.

Figure 1: Core workflow architecture of the ReMix prototype.

Figure 2 (Benchmark Performance Plot):
A comparative scatter/line plot showing KL divergence decay over 300 optimization iterations across Normal mixtures, Laplace mixtures, and mixed component families.

Figure 2: Optimization convergence across different mixture families.

4 Score-Function Estimator with Sticking the Landing (STL)

For the evidence lower bound (ELBO), we have $f_\theta(z) = \log p(x, z) - \log q_\theta(z)$. Then $d_\theta(z) = \partial_\theta f_\theta(z) = -s_\theta(z)$ and $\mathbb{E}_{q_\theta}[s_\theta(Z)] = 0$. Hence the score-function gradient baseline with the Roeder et al. Sticking the Landing (STL) modification is given by:

$$\hat{g}_{SF,STL}(Z) = (f_\theta(Z) - b)s_\theta(Z), \tag{2}$$

where $s_\theta(z) = \partial_\theta \log q_\theta(z)$ is the score, and b is a baseline independent of Z. For a general parameter-dependent objective f_θ, the term $d_\theta(Z)$ must be retained; deleting it is specific to

the ELBO. For softmax logits α_k where $\pi_k = \frac{e^{\alpha_k}}{\sum_j e^{\alpha_j}}$, the derivative with respect to the logits simplifies cleanly using responsibilities $r_j(Z)$:

$$\hat{g}_{\alpha_j,SF,STL} = (f_\theta(Z) - b)(r_j(Z) - \pi_j). \tag{3}$$

In our PyTorch implementation, this is elegantly constructed using detached evaluations and surrogate losses:

```
z = q.sample(torch.Size([self.n_samples])).detach()
log_q = q.log_prob(z)
values = objective.evaluate(q, z)
surrogate = (values.detach() - self.baseline) * log_q
return EstimateResult(values.detach().mean(), surrogate.mean())
```

5 Benchmark and Empirical Results

We evaluated our prototype on three distinct scenarios starting from initial weights $(0.5, 0.5)$ toward target weights $(0.2, 0.8)$ using the Adam optimizer ($lr = 0.05$, 300 iterations, 512 samples per iteration):

1. **Gaussian Mixture**: $\mathcal{N}(-2, 1)$ and $\mathcal{N}(2, 1)$. Estimated KL decreased from $0.2001 \rightarrow 0.0000$. Learned weights: $[0.193, 0.807]$.
2. **Laplace Mixture**: Laplace$(-2, 1)$ and Laplace$(2, 1)$. Estimated KL decreased from $0.1624 \rightarrow 0.0003$. Learned weights: $[0.188, 0.812]$.
3. **Mixed Family**: $\mathcal{N}(-2, 1)$ and Laplace$(2, 1)$. Estimated KL decreased from $0.1755 \rightarrow 0.0001$. Learned weights: $[0.192, 0.808]$.

6 Conclusion and Future Work

We have successfully built a working prototype of **ReMix**, combining arbitrary mixture component distributions, ELBO objectives, and standard PyTorch optimization loops. In subsequent phases of the project, we will integrate stratified, implicit reparameterization, and post-stratified estimators into this shared interface.

References

[1] J. Burroni and D. Sheldon. Gradient estimation for mixture variational inference. *Journal of Machine Learning Research*, 27(191):1-34, 2026.

[2] G. Roeder, Y. Wu, and D. K. Duvenaud. Sticking the landing: Simple, lower-variance gradient estimators for variational inference. *NeurIPS*, 2017.

[3] S. Mohamed et al. Monte Carlo gradient estimation in machine learning. *JMLR*, 21(132):1-62, 2020.